

# Thai Poetry Rhyme Verification Using a Hybrid Rule-Based Approach for Education and GenAI

**Abstract**—In Thai Klon-Paed (กลอนแปด) poetry (the eight-syllable verse form) each four-line stanza must satisfy an interlocking rhyming pattern where the rhyming syllables must share both a vowel sound (sàrà สระ) and a final-consonant class (mâ:ttra: มาตราตัวสะกด). Because Thai is written without word boundaries and its orthography conceals vowels and silenced letters, automatic rhyme verification is non-trivial: neither classical rule-based checkers nor G2P-romanization checkers handle the resulting edge cases reliably. We present a hybrid rule-based verification approach: first, we improve the KhaveeVerifier algorithm—vowel analysis, final-consonant-class analysis, the rhyme test, karun silencing, and true-final detection—and merge it into PyThaiNLP main (Model B); second, we introduce Klon8Checker (Model D), which augments these rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, recovering phonetic blind spots of the base oracle. We evaluate five checker configurations on a gold corpus of 36,475 classical stanzas (145,709 rhyme checks) and on a targeted augmentation of 11,623 instances: 10,000 adversarial negatives, 1,200 tricky positives, and 423 oracle-blind rhymes. The best configuration (D\_ssg) reaches 88.5% gold recall and 86.9% F1 on the augmentation, Model B attains 100% precision on curated negatives, and D\_w2p recovers 95.7% of the oracle-blind rhymes. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based Klon-Paed generation.

**Index Terms**—Thai NLP, Thai poetry, Klon-Paed, rhyme verification, rule-based systems, grapheme-to-phoneme, reinforcement learning, educational technology

## I. INTRODUCTION

Thai orthography does not map straightforwardly onto pronunciation. This gap becomes especially apparent in the classical register used by Thai poetry. The script is written without explicit word boundaries; vowels are frequently implicit; consonants or entire clusters can be silenced by the karun diacritic (th<sup>h</sup>ant<sup>h</sup>ák<sup>h</sup>â:t ท้ายจากมาตร); and irregular words whose pronunciations do not follow the standard phonetic rules are common. Complex phonological structures, such as consonant blending (k<sup>h</sup>am k<sup>h</sup>uâp klâm คำควบกล้ำ) and leading consonants (?âksôn nam อักษรนำ), further obscure syllable boundaries, vowel classification, and tone. As a result, syllabification and pronunciation are particularly difficult for computational algorithms and machine learning models.

Klon-Paed is a Thai verse form that depends on these phonetic properties. Each stanza (bòt บท) consists of four sets of seven-to-nine-syllable lines (vák วรรค) (wak). A valid rhyme requires two syllables to share both the vowel sound (sàrà สระ) (sara) and the final-consonant class (mâ:ttra: มาตราตัวสะกด). Four canonical rhyme rules must hold within (r1, r2, r3) and across (rX) stanzas (Section II), and the final syllable of every

wak must participate as depicted in Figure 1. A single silent letter, e.g. the silent (r ๖) of phet (เพชร) ('diamond'), or a short-long vowel-length error is enough to break a rhyme. These constraints are strict and deterministic, which is difficult for statistical systems.

A reliable automatic verifier is needed in two communities. In *education*, teachers and students need deterministic, explainable, hallucination-free feedback: an LLM that invents an incorrect rhyme should not be used in a classroom. In *NLP research*, training a language model to compose Klon-Paed, e.g. via reinforcement learning (RL), requires a programmatic reward that scores rhyme correctness. Because LLMs are probabilistic, they are unreliable at strict constraints such as syllable counts and rhymes, which a rule-based verifier supplies deterministically.

Existing verification systems fall short of this standard. Across five checker configurations (A, B, C, D\_w2p, D\_ssg), the prior KhaveeVerifier in PyThaiNLP 5.0.1 (Model A) relies on dictionary subword tokenization, omits one canonical rhyme rule, and reaches only 73.4% recall on our gold corpus. G2P romanization-based checkers, such as the word\_check grader of KlonSuphap-LM [11] (Model C), collapse long/short vowel distinctions and produce thousands of false positives on adversarial negatives. In this paper we address these limitations with a hybrid rule-based approach and a rigorous, adversarial evaluation. Our contributions are:

- **An improved rule-based verifier (Model B).** We rework the vowel analysis (check\_sara), the final-consonant-class analysis (check\_marttra), the rhyme test (is\_sumpus), karun silencing, and true-final detection, and implement all four canonical rhyme rules. The result is merged into PyThaiNLP main (4 Aug 2026) and is set to ship as the default KhaveeVerifier in PyThaiNLP 6.0.
- **A dictionary-enhanced variant (Klon8Checker, Model D).** We augment the rules with a gold-standard G2P override dictionary of over 4,000 entries and a fallback segmenter, recovering the oracle's blind spots.
- **A large evaluation corpus.** 36,475 gold stanzas (145,709 rhyme checks) from five classical works, plus 11,623 augmentation instances: 10,000 oracle-verified negatives, 1,200 tricky positives, and 423 oracle-blind positives.
- **A systematic comparison.** Five configurations (A, B, C, D\_w2p, D\_ssg) are compared on precision, recall, F1, and coverage, with per-rule and per-operator error analyses.
- **Educational tool and RL-ready reward.** An explainable web tool plus a deterministic reward signal for RL-based

## II. BACKGROUND AND RELATED WORK

### A. The Linguistic Rules of Klon-Paed

A Klon-Paed stanza has four waks of seven-to-nine syllables each. Thai syllables are written without delimiters and a word may consist of one or more syllables, so the meter must be recovered by syllabification rather than by counting words. Within this skeleton the verse follows a fixed rhyme scheme. As defined by the Royal Institute of Thailand, two syllables rhyme (*sumpus* สัมผัส) when they share the same vowel sound (sàrà สระ) and the same final-consonant class (mâ:ttra: มาตราตัวสะกด). Additionally, tone is not part of the rhyme. The matra classes (mê: เมี), consisting of ka, kong, kok, kot, kon, kop, kom, koey, and koew, group the final consonants of Thai syllables by their phonetic outcome (the velar, dental, labial, nasal, semivowel, and (v ɹ) finals). The four waks are called the (sàdàp สดับ), (ráp รัน), (rɔng รอง), and (sòng ส่ง) respectively. Four canonical rhyme rules are checked in this work, following the *klon* canon and the implementation of the PyThaiNLP KhaveeVerifier:

- r1 (sàdàp → rap): the final syllable of wak 1 must rhyme with one of the first five syllables of wak 2 (syllables 3 and 5 are obligatory, 1, 2, and 4 are permissible);
- r2 (rap → rong): the final syllable of wak 2 must rhyme with the final syllable of wak 3;
- r3 (rong → song): the final syllable of wak 3 must rhyme with one of the first five syllables of wak 4 (syllables 3 and 5 are obligatory, 1, 2, and 4 are permissible);
- rX (inter-stanza): the final syllable of wak 4 of the previous stanza rhymes with the final syllable of wak 2 of the current stanza.

As a concrete example, the well-known stanza of the *Phukao thong* (นิราศภูเขาทอง) travel poem by Sunthorn Phu satisfies r1–r3; Figure 1 shows its rhyme positions and the inter-stanza (rX) link to the preceding stanza.

Three orthographic phenomena make these rules hard to apply automatically. First, compound, transformed, and reduced vowels (sàrà pràsóm สระประสม), (sàrà plian.rù:p สระเปลี่ยนรูป), (sàrà lôt.rù:p สระลดรูป) let several spellings map to one vowel sound. Second, the karun diacritic silences a letter or a whole cluster, leaving unpronounced letters in the written form. Third, a trailing y, l, w (ย, ล, ว) may be a genuine final consonant or merely part of a vowel digraph or an initial cluster. Any verifier must resolve all three before it can compare vowel and final class.

### B. Thai Text-Processing Foundations

Two building blocks underlie all Thai NLP systems: segmentation and grapheme-to-phoneme conversion. Because Thai has no word delimiters, segmentation has received sustained attention: dictionary-based maximum matching (the newmm engine of PyThaiNLP [1]), neural segmenters such as DeepCut [5] and AttaCut [3], syllable-based neural segmentation [4], and the SSG syllable segmenter shipped in

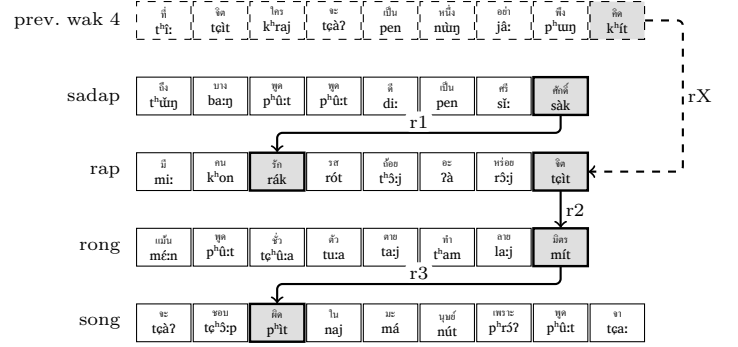


Fig. 1. The example stanza with its rhyme positions shaded and linked: r1 (sadap→rap), r2 (rap→rong), r3 (rong→song); the dashed rX arrow links the final syllable (kʰit) of the previous stanza’s wak 4 to wak 2’s final syllable (tɛit).

PyThaiNLP 5.x, used throughout this work. G2P conversion maps surface orthography to pronunciation. Thai G2P has been addressed with rule-based (TLTK [6]), example-based [7], CRF joint-sequence [8], and two-stage sequence [9] models. G2P is directly relevant to rhyme verification because rhyme is a property of *sound*, not of *spelling*.

### C. Computational Poetry Analysis and Generation

Prior work on Thai poetry in NLP has concentrated on analysis and generation. On the analysis side, machine-learning studies of Sunthorn Phu’s verse have extracted sound patterns such as rhyme, assonance, and alliteration using internal-rhyme rules [10]. On the generation side, the dominant line of work fine-tunes language models on corpora of classical verse: the Phra Aphai Mani LM [12] and its successor KlonSuphap-LM [11] fine-tune a GPT-2 Thai base model, then use *rhyme tagging* (annotating the tokens that must rhyme), attention masking, and finally reinforcement learning in which a rule-based *klon grader* supplies the reward. That grader is the word\_check module evaluated here as Model C, making our evaluation directly relevant to the RL step of that pipeline. More broadly, RLHF has established reward models as essential for aligning LLMs to follow strict requirements [2], and grammar-constrained decoding enforces structural constraints at inference time [13].

### D. Positioning of This Work

Compared with this landscape, our contribution is the first systematic, adversarially augmented evaluation of Thai rhyme *verification* as opposed to generation: Model A has never been stress-tested against curated edge cases, the G2P-based grader (Model C) has not been validated for precision, and no open verifier covers all four rhyme rules with dictionary support for irregular pronunciations. We fill this gap with improved rule-based verification (Model B), a dictionary-enhanced variant (Model D), and a large gold-with-augmentation evaluation, including a novel *oracle-blind* probe that holds the verifier accountable for rhymes its oracle cannot see.



TABLE I  
PRINCIPAL CHANGES OF THE 6.0 KHAVEEVERIFIER OVER THE 5.0.1 BASELINE.

| Aspect                                                     | 5.0.1 → 6.0 (example)                                                                           |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Segmentation                                               | dict subword → SSG syllable segmentation                                                        |
| Karun ( <sup>h</sup> ant <sup>h</sup> ák <sup>h</sup> ā:t) | ignored → silenced (ohm→o)                                                                      |
| True finals y/l/w                                          | naive → cluster-aware (thai, klai→mae ka)                                                       |
| Special-vowel norm.                                        | one-sided → both-sided (cam~kam)                                                                |
| Compound/ reduced vowels                                   | misread → compound-vowel classifier, short-vowel mark, and ua diphthong (cet→short e; tua~chua) |
| <i>rú</i>                                                  | one sound → three sounds ( <i>rā:k</i> , <i>rít</i> , <i>rú.du</i> )                            |
| r3 rule                                                    | absent → implemented (all four rules)                                                           |
| Silent r                                                   | misread → stripped in <i>check_marttra</i> (but, phet)                                          |
| Single-char words                                          | crash → guarded (na, tha→a)                                                                     |
| Silent terminal vowels                                     | misread → stripped ( <i>kiàt</i> , <i>té<sup>h</sup>àt</i> )                                    |
| Stanza processing                                          | flat → stanza-based, word counts, structured errors                                             |

labelled hints and never trusted on their own. At inference time Klon8Checker segments the wak into words with an override-augmented newmm dictionary, looks each up in the override dictionary, and syllabifies unmatched words with the fallback segmenter. We report two configurations: D\_w2p, which falls back to the w2p neural G2P model, and D\_ssg, which falls back to plain SSG segmentation; as Section V shows, the w2p fallback hallucinates on short or compound words, making D\_ssg the more robust configuration.

### C. Baseline Systems

Model A is the original KhaveeVerifier of PyThaiNLP 5.0.1, instrumented verbatim from the vendored source: dictionary-based subword\_tokenize, no r3 rule, no karun handling, no true-final logic. Model C is the word\_check rhyme grader of KlonSuphap-LM [11]: each wak is romanized with the TLTK G2P model and the romanized vowels and finals compared. Its two documented weaknesses are (i) a replace\_long\_short step that collapses long and short vowels and (ii) a mattra comparison that ignores the final consonant, both inflating recall at the cost of precision; tlk failures reduce coverage.

## IV. DATASET CONSTRUCTION AND AUGMENTATION

### A. Gold-Standard Corpus Compilation

We collected five classical Thai narrative poems from the Vajirayana digital library [14]: *k<sup>h</sup>o:bùt* (โคบุตร), *k<sup>h</sup>ũn tē<sup>h</sup>á:ŋ* *k<sup>h</sup>ũn p<sup>h</sup>ē:n* (ขุนช้างขุนแผน), *p<sup>h</sup>rā ?àp<sup>h</sup>ai máni*: (พระอภัยมณี), *nírá:t* *p<sup>h</sup>u:k<sup>h</sup>áo t<sup>h</sup>o:ŋ* (นิราศภูเขาทอง), and *sùp<sup>h</sup>asit sǒ:n jǐŋ* (สุภาษิตสอนหญิง). A cleaning pipeline normalised the text and split it into stanzas of four waks, discarding only chapter-opening fragments and scribal *o* marks. Crucially, waks with ten or more syllables were retained: dropping them severs inter-stanza rhyme chains and made an early corpus version appear to violate rX at a

72% rate, whereas on the complete corpus rX holds at 96–97%. The final corpus comprises 191 files, 36,475 stanzas, 145,900 waks, and 145,709 gold rhyme checks; the gold is all-positive, because the classical poems are presumed to rhyme. The Thai Literature Corpora [15] cover similar material.

### B. Targeted Data Augmentation

Recall over an all-positive corpus cannot measure precision, and easy positives cannot expose blind spots. We therefore augmented the gold with adversarial instances generated by an automated pipeline. Every instance is *standalone*: it is derived from a real stanza but only the target rule is changed, so exactly one rule is exercised while the others remain intact. Every negative is verified by the 6.0 oracle (is\_sumpus) to be a genuine non-rhyme, and the whole set was audited in four independent passes in which reviewers spelled out the sara and mattra of every candidate (with vowel length made explicit) and double-checked pronunciations against the IPA. The audit removed 56 candidate words the oracle misreads—karun clusters it fails to silence and multi-syllable Pali words SSG keeps as single tokens.

a) *Negatives (precision probes)*.: The 10,000 negatives are drawn from eight corruption operators: trivial different-vowel-and-matra swaps (C6, 300), same-matra swaps (C0, 500), same-vowel swaps (C1, 500), short/long vowel swaps (C3, 3,100), liquid-final swaps (C4, 1,200), a systematic trap in which the 5.0.1 checker reads the candidate as rhyming with the target while 6.0 does not (C9, 2,800), leading-consonant words (C5, 600), and karun/*ru* (ฤ)/cluster traps (C2, 1,000). C3 and C9 dominate the mix because they are the operators that best discriminate the checkers.

b) *Positives (recall probes)*.: The 1,200 *tricky* positives swap the target syllable with a rhyming candidate that exercises normalization (*sàrà plian.rú:p/lót.rúp* สระเปลี่ยนรูป/ลดรูป), *rú* (ฤ), karun, and true-final cases; all are oracle-confirmed rhymes. A further 423 *oracle-blind* positives are genuine rhymes the oracle labels as non-rhymes: the silent-*r* *p<sup>h</sup>ét* (เพชร) (true *e*:+kòt, which the oracle reads with the long *ae* because the dead final carries no short-vowel mark), and the first-sara words *sàttru*: (สัตว์), *kàsàttri*: (กษัตริ), and *kàsàttri*: (กษัตริย์) (SSG keeps each as one token, so check\_sara hears only the first vowel). For these the gold label is the *linguistic* truth, not the oracle verdict: a checker that hears the real rhyme receives credit, while the oracle itself (Model B) is charged a false negative—“the oracle that is wrong loses points.”

## V. EVALUATION AND RESULTS

### A. Experimental Setup

We report precision, recall, and F1 with Wilson 95% confidence intervals, coverage, and a conservative drop-as-fail variant for Model C. Because every augmentation instance is *standalone* and tests exactly one rule, the per-rule prediction is the clean metric; a stanza-level aggregate over the augmentation is only well-defined for Model B (the oracle). Model A has no r3 and is reported as N/A there. All checkers were instrumented from verbatim vendored sources (the 5.0.1 and

TABLE II  
GOLD RECALL (STANZA LEVEL AND PER RULE), 36,475 STANZAS.

| System      | Stanza            | r1   | r2   | r3   | rX   |
|-------------|-------------------|------|------|------|------|
| A (5.0.1)   | 73.4              | 86.3 | 88.8 | N/A  | 88.7 |
| B (6.0)     | 87.5              | 94.7 | 96.5 | 95.5 | 96.7 |
| D_w2p       | 87.4              | 95.4 | 96.3 | 94.9 | 96.5 |
| D_ssg       | <b>88.5</b>       | 95.9 | 96.7 | 95.2 | 97.0 |
| C (Kongfha) | 84.8 <sup>†</sup> | 93.2 | 96.5 | 94.1 | 96.6 |

<sup>†</sup> coverage 95.7%; per-rule figures on the evaluated subset.

6.0 cores), with Model C run under Python 3.12 through a persistent pool of ten tlk workers (the full augmentation pass takes about 40 minutes; A/B/D are parallelised and finish in under a minute).

### B. Gold-Corpus Recall

Table II reports stanza-level and per-rule recall over the 36,475 gold stanzas. The proposed family dominates: D\_ssg reaches 88.5% stanza recall and B 87.5%, versus 73.4% for Model A. Model C reaches 84.8% but evaluates only 95.7% of the corpus (tlk failures concentrate in the archaic Khun Chang Khun Phaen).

### C. Augmentation: Precision, Recall, and F1

Table III summarises the augmentation-only results (pooled per rule). Three results stand out.

First, Model B is a perfect oracle on the negatives: zero false positives across all 10,000 (100% precision by construction). Its recall is limited only by the oracle-blind positives it cannot see (73.9%).

Second, Model D recovers most of those blind spots: D\_w2p reaches 92.1% recall and D\_ssg 87.0%, with precision 83.4% and 86.8%, giving the best augmentation F1 (87.5% for D\_w2p). Table IV shows the mechanism: D\_w2p hears 405 of the 423 oracle-blind rhymes (95.7%), D\_ssg 288 (68.1%), B none.

Third, the baselines invert the precision/recall trade-off. Model C is recall-strong (87.1% on the augmentation, 78.7% even on oracle-blind rhymes, because its G2P romanization hears the real sound) but makes 3,107 false positives—2,702 on the short/long vowel swap (C3), which its `replace_long_short` collapses—for 30.4% precision. Model A is weak on both sides (28.4% precision, 62.7% recall), with 1,774 of 1,950 false positives from the C9 old-accept trap. Per rule, B keeps 100% precision on all four rules (r1 recall lowest at 54.5%, rX highest at 98.0%), C’s precision collapses on every rule (24–35%), and D\_ssg stays above 75% precision and 82% recall throughout.

### D. Oracle-Blind Probe and Error Analysis

Table IV reports the oracle-blind probe with the combined false-negative accounting (tricky + oracle-blind) and the FP breakdown by the two most informative operators. The combined FN-pos column is the headline: B carries 423 false negatives (all oracle-blind), A carries 848, and D carries only 128 (D\_w2p) or 211 (D\_ssg). C’s precision collapse

TABLE III  
AUGMENTATION-ONLY RESULTS (POOLED PER RULE, 11,623 INSTANCES). FP = FALSE POSITIVES ON 10,000 NEGATIVES; FN-POS = COMBINED FALSE NEGATIVES ON 1,623 POSITIVES.

| System      | P            | R           | F1          | FP       | FN-pos |
|-------------|--------------|-------------|-------------|----------|--------|
| A (5.0.1)   | 28.4         | 62.7        | 39.1        | 1,950    | 848    |
| B (6.0)     | <b>100.0</b> | 73.9        | 85.0        | <b>0</b> | 423    |
| D_w2p       | 83.4         | <b>92.1</b> | <b>87.5</b> | 298      | 128    |
| D_ssg       | 86.8         | 87.0        | 86.9        | 215      | 211    |
| C (Kongfha) | 30.4         | 87.1        | 45.1        | 3,107    | 266    |

TABLE IV  
ORACLE-BLIND PROBE AND ERROR ANALYSIS (FN-POS = TRICKY + ORACLE-BLIND).

| System | recall (%) |              | FN-pos     |     | FP       |           |
|--------|------------|--------------|------------|-----|----------|-----------|
|        | tricky     | oracle-blind | total      | =ob | total    | C3/C9     |
| A      | 57.0       | 21.5         | 848        | 332 | 1,950    | 162/1,774 |
| B      | <b>100</b> | 0.0          | 423        | 423 | <b>0</b> | 0/0       |
| D_w2p  | 90.8       | <b>95.7</b>  | <b>128</b> | 18  | 298      | 135/103   |
| D_ssg  | 93.7       | 68.1         | 211        | 135 | 215      | 119/48    |
| C      | 85.3       | 78.7         | 266        | 90  | 3,107    | 2,702/267 |

concentrates on the short/long vowel swap (C3) and A’s on the old-accept trap (C9).

### E. Discussion

The experiment supports four findings. (1) **The proposed rules are precise.** B never false-accepts on oracle-verified negatives, and B, D\_w2p, and D\_ssg all exceed 99% merged precision (A 93%, C 91%). (2) **The override dictionary recovers the oracle’s blind spots.** The D\_w2p-versus-D\_ssg gap on oracle-blind recall (95.7% versus 68.1%) is the pronunciation-knowledge contribution: the overrides read `เพชร` as *p<sup>h</sup>ét* and split `ศักร` and `กษัตริย์` into their true syllables *sàt.tru:* and *kà.sàt.tri:*, respectively. One limitation is that each entry forces a single canonical pronunciation, although many surface forms are heteronyms whose reading depends on context, and in natural delivery adjacent syllables are often merged so that a rhyme lands. The current entries are handcrafted: they originate from Word2Phrase (w2p) candidate readings that were verified manually, and no public G2P corpus has been used yet. Mining a gold-standard G2P corpus to expand the dictionary is future work to improve coverage and accuracy. (3) **C trades precision for recall on exactly the vowel-length distinction** the rule-based family enforces (78.7% oracle-blind recall, but 30.4% precision). (4) **The oracle itself has a blind spot** that only the oracle-blind probe exposes: a purely oracle-supervised evaluation would report B as flawless. On the merged gold+augmentation set, F1 favours D\_ssg (93.6%), D\_w2p (93.0%), and B (93.0%), with C (87.8%) and A (82.0%) behind. The discriminating precision lives in the augmentation-only tables.

## VI. APPLICATIONS AND FUTURE WORK

### A. Educational Impact

The verifier is deployed as an interactive web tool for Klon 4/8 teachers and students: it segments a poem into syllables, highlights rhyme positions, spells out the sara and mattra of every syllable, and explains each error in terms of the four rhyme rules. The rule-based feedback is reproducible and hallucination-free. Future work includes classroom evaluations and pronunciation guidance for irregular words.

### B. Foundation for Generative AI

The same determinism makes the verifier a suitable reward signal for fine-tuning small, locally run LLMs: exact, immediate, and free of the noise of a learned reward model. It mirrors the RL step of the KlonSuphap-LM pipeline [11], but with a grader (Model B or D) substantially more precise than the G2P-based grader used there (Model C), particularly on vowel length. Fine-tuning a generator with this reward, scaling the override dictionary from a public gold-standard G2P corpus, and handling heteronyms and on-the-fly syllable merging are future work, alongside decoding-time constrained sampling [13].

## VII. CONCLUSION

We presented a hybrid rule-based approach to Thai Klon-Paed rhyme verification. An improved rule-based verifier (Model B), now the default *KhaveeVerifier* in *PyThaiNLP* main and to ship in 6.0, implements all four canonical rhyme rules with robust karun, true-final, and sara handling and reaches 100% precision on curated negatives. A dictionary-enhanced variant (Klon8Checker, Model D) adds a gold-standard G2P override dictionary of over 4,000 entries and recovers 95.7% of the oracle’s own blind spots. On 36,475 gold stanzas and 11,623 adversarial instances, the proposed family raises stanza recall from 73.4% to 88.5% ( $D_{\text{ssg}}$ ) with near-perfect precision, exposing the blind spots of both the G2P-romanization checker and the rule-based oracle. The verifier is released as an explainable educational tool and provides a deterministic reward signal for RL-based generation; its integration into *PyThaiNLP* makes the improvement available to the entire Thai NLP ecosystem.

## ACKNOWLEDGMENT

We thank the Vajirayana Digital Library [14], the *PyThaiNLP* maintainers for merging the *KhaveeVerifier* improvements, and the TLTK and KlonSuphap-LM developers.

## REFERENCES

- [1] *PyThaiNLP* Contributors, “*PyThaiNLP*: Thai natural language processing in Python,” 2024. [Online]. Available: <https://github.com/PyThaiNLP/pythainlp>
- [2] L. Ouyang, J. Wu, X. Jiang, et al., “Training language models to follow instructions with human feedback,” in *NeurIPS*, 2022.
- [3] P. Chormai, P. Prasertsom, and A. T. Rutherford, “AttaCut: A fast and accurate neural Thai word segmenter,” *arXiv preprint arXiv:1911.07056*, 2019.
- [4] P. Chormai, P. Prasertsom, J. Cheevaprawatdomrong, and A. T. Rutherford, “Syllable-based neural Thai word segmentation,” in *Proc. 28th International Conference on Computational Linguistics (COLING)*, 2020.
- [5] T. Kittinaradorn, et al., “DeepCut: A Thai word tokenization library using deep neural network,” 2019. [Online]. Available: <https://github.com/rkcosmos/deepcut>
- [6] W. Aroonmanakun, “TLTK: Thai language toolkit,” [Online]. Available: <https://pypi.org/project/tltk/>
- [7] P. Charoenpornasawat and T. Schultz, “Example-based grapheme-to-phoneme conversion for Thai,” in *Proc. Interspeech*, 2006.
- [8] S. Saychum, S. Kongyoung, A. Rugchatjaroen, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, “Efficient Thai grapheme-to-phoneme conversion using CRF-based joint sequence modeling,” in *Proc. Interspeech*, 2016.
- [9] A. Rugchatjaroen, S. Saychum, S. Kongyoung, P. Chootrakool, S. Kasuriya, and C. Wutiwiwatchai, “Efficient two-stage processing for joint sequence model-based Thai grapheme-to-phoneme conversion,” *Speech Communication*, 2019.
- [10] P. Suksanguan, S. Waijanya, and N. Promrit, “The extraction of beautiful sound patterns from Sunthorn Phu’s poem using machine learning technique and internal rhyme rule,” 2021.
- [11] K. Yingyingseree, “KlonSuphap-LM: GPT-2 for Thai Klon-Paed,” *Hugging Face model*, 2024. [Online]. Available: <https://huggingface.co/Kongfha/KlonSuphap-LM>
- [12] K. Yingyingseree, “PhraAphaiManee-LM,” *Hugging Face model*, 2023. [Online]. Available: <https://huggingface.co/Kongfha/PhraAphaiManee-LM>
- [13] S. Geng, M. Josifoski, M. Peyrard, and R. West, “Grammar-constrained decoding for structured NLP tasks without fine-tuning,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [14] Vajirayana Digital Library. [Online]. Available: <https://vajirayana.org>
- [15] A. Rutherford, “Thai literature corpora (TLC),” [Online]. Available: <https://attapol.github.io/tlc.html>